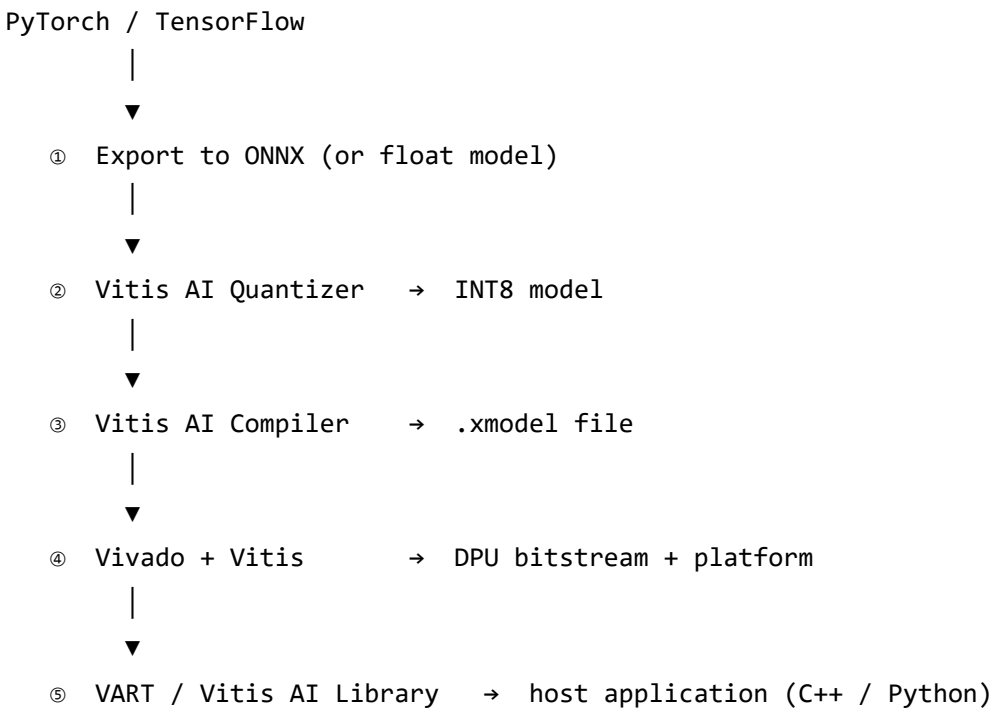
Standard AMD Workflow to deploy AI model on an FPGA

The AMD ecosystem provides four major tools for deploying AI on FPGA/SoC hardware:

Tool	Role
Vitis AI	AI-model quantization, compilation, and runtime (the AI-specific pipeline)
Vitis HLS	High-Level Synthesis — C/C++ → RTL for custom operators the DPU cannot handle
Vivado	Traditional FPGA design: RTL synthesis, place-and-route, IP integration, bitstream generation
Vitis	Unified software platform: host application, PS–PL linking, platform creation, system debug

> **Typical target hardware:** AMD Zynq MPSoC / Zynq UltraScale+ (e.g., ZCU102, ZCU104, Kria SOM) and Versal ACAP boards. These combine an ARM processing system (PS) with FPGA programmable logic (PL), allowing the DPU to sit in the PL while Linux and VART run on the PS.

Workflow Overview



This is the canonical AMD Vitis AI workflow.

Step 1 — Train & Export

- Train your model in **PyTorch** or **TensorFlow** (float32). - Export the trained model to **ONNX** (Open Neural Network Exchange), or pass the float model directly into Vitis AI if the framework is natively supported. Vitis AI accepts both ONNX and native TensorFlow/PyTorch formats.

Step 2 — Quantization (Vitis AI Quantizer)

- Convert weights and activations from **float32 → INT8**. - Vitis AI performs **post-training quantization (PTQ)** using a small calibration dataset (100–1000 unlabeled samples) to analyse activation distributions. - If accuracy loss is unacceptable, **quantization-aware training (QAT)** can fine-tune the model with simulated quantization — use **Brevitas** (PyTorch) or **QKeras** / **TensorFlow Model Optimization**.

Step 3 — Compilation (Vitis AI Compiler)

- Feed the quantized model into the **Vitis AI Compiler**. - The compiler maps the model graph onto the target DPU architecture (specified by an `arch.json` file). It performs operator fusion (e.g.,

BatchNorm folded into Convolution), data-layout transformations, and instruction scheduling across DPU cores. - Output: a `.xmodel` file — the compiled instruction stream the DPU executes at runtime.

Step 4 — Hardware Platform (Vivado + Vitis)

This is the hardware-design step that creates the FPGA bitstream containing the DPU:

1. **Vivado** — integrate the **DPU IP core** into a block design. Configure the DPU (number of cores, BRAM/URAM allocation, supported convolution types). Add other necessary IP: clocking, resets, AXI interconnects, MIPI/CSI camera receivers, etc. Run synthesis, implementation (place-and-route), and generate the **bitstream** (`.bit`) and hardware hand-off file (`.xsa`). 2. **Vitis** — import the `.xsa` from Vivado to create a **platform project**. The platform packages the hardware design with the software stack (FSBL, PMU firmware, Linux kernel, device tree) so the host application can communicate with the DPU.

> **Vitis HLS** (optional / advanced): If your model contains operators the DPU does not natively support, write a custom accelerator in C/C++, synthesize it to RTL via Vitis HLS, and integrate the resulting IP block into the Vivado design alongside the DPU. For most standard CNNs (ResNet, YOLO, MobileNet, VGG, etc.) this is unnecessary — the DPU handles them directly.

Step 5 — Deployment & Host Application

Flash the Bitstream

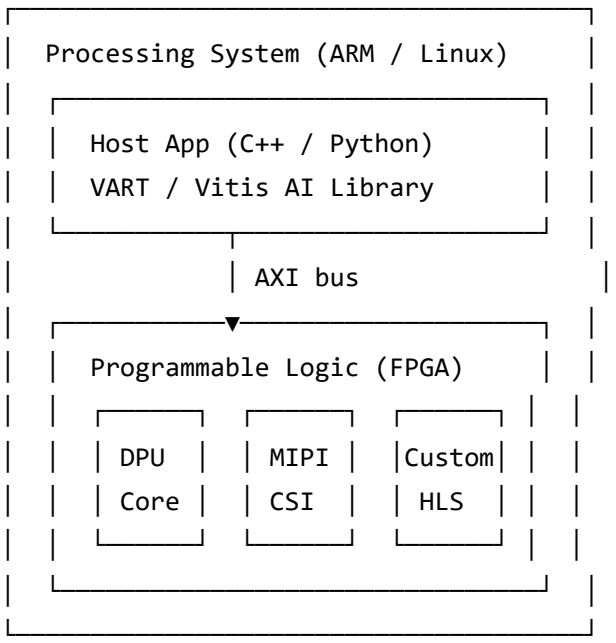
Load the DPU bitstream onto the board. Common methods:

Method	Use Case
Vivado Hardware Manager (JTAG over USB)	Development / debugging
SD card / eMMC boot	Standalone embedded deployment (Zynq MPSoC, Kria)
QSPI flash (`program_flash` via Vivado)	Production, non-volatile boot
Ethernet / PCIe	Data-centre accelerator cards (Alveo)

Write the Host Application

On the ARM processing system, use the **Vitis AI Runtime (VART)**:

- **C++ or Python API** to load the `.xmodel`, submit inference jobs to the DPU asynchronously, and retrieve results. - **Vitis AI Library** provides higher-level wrappers with pre-built pre-processing (resize, normalise) and post-processing (NMS, softmax) pipelines for common tasks. - Typical loop: capture frame from camera (MIPI) → pre-process → DPU inference → post-process → act on result.



Accurate PS/PL architecture diagram for Zynq/Versal platforms.

Tool Summary

Step	Tool	Input	Output
Train	PyTorch / TensorFlow	Dataset	float32 model
Export	`torch.onnx` / `tf2onnx`	float32 model	ONNX
Quantize	Vitis AI Quantizer	float32 model + calibration data	INT8 model
Compile	Vitis AI Compiler	INT8 model + `arch.json`	`.xmodel`
Hardware	Vivado	DPU IP + block design	bitstream (`.bit`) + `.xsa`
Platform	Vitis	`.xsa`	platform (boot images, device tree)

Step	Tool	Input	Output
Program	Vivado Hardware Manager / SD boot	bitstream	running DPU on FPGA
Host App	VART / Vitis AI Library	`.xmodel` + sensor data	inference results

> **Environment note:** Vitis AI tools ship as **Docker containers** (CPU and GPU variants). Vivado and Vitis are native Linux/Windows installs. The DPU IP core is included with Vitis AI and instantiated inside Vivado.

External Resources

- Advanced FPGA Course: Verilog Based Robotics & Signal Processing
- hls4ml
  - translate machine learning models directly into synthesizable VHDL or Verilog
- PyTorch AI on FPGA — FINN Workflow Tutorial
- Python to FPGA
- Educational Platform for FPGA Accelerated AI in Robotics
- Workflow of deploying a model (AMD) — Vitis AI 3.5 docs

AI Module Comparison

Platform / Device	Peak AI Performance (TOPS)	Power Consumption	Primary Use Case & Architecture
NVIDIA Jetson AGX Orin	Up to 275 TOPS (INT8)	15W – 60W	High-end robotics, autonomous navigation. Integrates CPU, Ampere GPU, and Deep Learning Accelerators (DLA).
NVIDIA Jetson Orin Nano (Super)	Up to 67 TOPS (INT8, 8 GB SKU) / 40 TOPS (original non-Super)	7W – 25W	Entry-level embedded AI and smart cameras. The "Super" variant (Dec 2024) offers a significant TOPS uplift over the original Orin Nano. Lower-power CUDA entry point.

Platform / Device	Peak AI Performance (TOPS)	Power Consumption	Primary Use Case & Architecture
Raspberry Pi 5 + Hailo-8 AI HAT	~13 to 26 TOPS	10W – 15W	Low-power field robots. Relies on a separate Hailo accelerator (Hailo-8L = 13 TOPS, Hailo-8 = 26 TOPS) for edge inference.
ASUS NUC 14	20–40+ TOPS (NPU + GPU, varies by generation)	28W – 65+W (system)	Industrial edge / edge server. Full x86 compatibility and PCIe expansion. **Generation breakdown:** Meteor Lake NPU ≈ 11 TOPS, Arrow Lake NPU ≈ 13 TOPS, Lunar Lake NPU ≈ 48 TOPS. Combined NPU + Arc GPU can reach 40+ TOPS on Lunar Lake. Specify the exact CPU generation when comparing.
Axelera AI Mini PC With M.2 Max	Up to 214 TOPS	20W – 40W	Based on in-memory computing (PIM) and RISC-V; optimized for ultra-low latency at high TOPS.
EdgeCortix SAKURA	Up to 60 TOPS	Under 10W	Highly efficient FIM (Fabric in Memory) architecture; targets low-power smart city and vision applications.
SiMa.ai MLSoC	Up to 50+ TOPS	~5–10W (typical board power)	Purpose-built for edge AI; integrated CV/vision pipeline with software-programmable NPU. Power varies by workload — peak throughput draws toward the higher end of the range. Verify against a specific SKU datasheet.